

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

Department of Information and Communication Technology

WEEKLY INTERNSHIP REPORT

Week Report - Implementation Phase

By

Bui Tat Dat

Student ID: Your Student ID

Cyber Security

Title

**Implementation Progress of a
Docker-based SOC Laboratory using
Wazuh, DFIR-IRIS, and crAPI**

Supervisor: Supervisor Name

Organization: Company / Internship Organization

Current status: Infrastructure implementation completed; detection
engineering is pending

Hanoi, 2026

Abstract

This weekly report summarizes the current implementation progress of a Docker-based Security Operations Center (SOC) laboratory built with Wazuh, DFIR-IRIS, and crAPI. The primary work completed this week is infrastructure implementation: the vulnerable API environment has been arranged into segmented Docker networks, Wazuh components have been prepared as the SIEM/XDR monitoring layer, DFIR-IRIS has been prepared for case management, and Wazuh agents have been mapped to the web, backend, and database layers for future log collection.

At the current stage, the project has **not yet performed penetration testing**, has **not yet written custom Wazuh decoders or detection rules**, and has **not yet collected attack-pattern logs**. Therefore, this report does not present detection results, rule accuracy, alert statistics, or attack evaluation metrics. Those tasks are intentionally moved to the next phase after the infrastructure is stable.

The report focuses on what has been implemented, how the lab is organized, what components are currently deployed, what limitations remain, and what work will be conducted next. This avoids overstating the project results and keeps the weekly report aligned with the actual implementation status.

Keywords: SOC Lab, Wazuh, DFIR-IRIS, crAPI, Docker Compose, API Security, Log Collection, Implementation Progress.

Contents

Abstract	i
List of Acronyms	iv
1 Weekly Progress Overview	1
1.1 Purpose of This Weekly Report	1

1.2	Completed Work This Week	1
1.3	Work Not Yet Completed	2
1.4	Current Project Status	2
2	Implemented Lab Architecture	4
2.1	Architecture Summary	4
2.2	Lab Components	4
2.3	Lab Simulation Status	5
3	Docker-based Implementation	6
3.1	crAPI Target Environment	6
3.2	Network Segmentation	6
3.3	Wazuh SIEM/XDR Stack	7
3.4	Wazuh Agent Mapping	7
3.5	DFIR-IRIS Case Management Stack	8
4	Current Validation and Limitations	9
4.1	Infrastructure Validation Performed	9
4.2	Known Implementation Issues	9
4.3	Current Limitations	10
4.4	Reason for Deferring Pentest and Rule Writing	10
5	SIEM Platform Comparison	11
5.1	Purpose of the Comparison	11
5.2	Comparison Analysis	12
6	Next Phase Plan	13
6.1	Planned Work for the Next Week	13
6.2	Planned Detection Engineering Workflow	13
6.3	Expected Future Attack Scenarios	14
7	Conclusion	15
A	Configuration Summary	16
A.1	Wazuh Manager Configuration Summary	16
A.2	Current Pending Items	16

List of Tables

- 1.1 Current implementation status 3
- 3.1 Implemented crAPI service grouping 6
- 3.2 Implemented Docker network zones 7
- 3.3 Wazuh components prepared in the current implementation 7
- 3.4 Wazuh agent and log source mapping 8
- 3.5 DFIR-IRIS components prepared for future case management 8
- 5.1 Comparison between Wazuh, Microsoft Sentinel, and Elastic Stack . . . 11

List of Figures

- 2.1 Implemented lab components for the Docker-based SOC laboratory. . . 5
- 2.2 Planned lab simulation workflow for the next phase. 5

List of Acronyms

Acronym	Meaning
API	Application Programming Interface
BAC	Broken Access Control
BOLA	Broken Object Level Authorization
DFIR	Digital Forensics and Incident Response
IDS	Intrusion Detection System
IRIS	Incident Response Investigation System
SIEM	Security Information and Event Management
SOC	Security Operations Center
XDR	Extended Detection and Response
DMZ	Demilitarized Zone

Chapter 1

Weekly Progress Overview

1.1 Purpose of This Weekly Report

The purpose of this report is to document the current progress of the SOC laboratory implementation. In the previous planning stage, the expected final project involved three major parts: infrastructure deployment, attack simulation, and detection engineering. However, in the current week, only the implementation and deployment part has been completed. The offensive testing and detection rule development phases are still pending.

Therefore, this report is rewritten to accurately reflect the actual progress. Instead of presenting penetration-testing results or custom detection rules that have not been completed yet, this version focuses on infrastructure readiness, Docker Compose structure, network segmentation, log-source planning, and next-step preparation.

1.2 Completed Work This Week

The completed work is summarized as follows:

- Implemented a Docker-based lab architecture for crAPI, Wazuh, and DFIR-IRIS.
- Organized crAPI services into logical network zones: external access, DMZ, backend, database, and SOC monitoring network.
- Prepared Wazuh Manager, Wazuh Indexer, and Wazuh Dashboard as the monitoring stack.
- Prepared DFIR-IRIS components including application, database, worker, RabbitMQ, and Nginx reverse proxy.

- Added Wazuh agent containers for web, backend, and database layers.
- Mapped application and database logs into shared Docker volumes for future collection.
- Identified the next implementation tasks required before detection evaluation.

1.3 Work Not Yet Completed

The following tasks are **not yet completed** and should not be presented as finished results:

- Penetration testing against crAPI.
- Broken Access Control or BOLA attack execution.
- Collection of attack-pattern logs.
- Writing custom Wazuh decoders.
- Writing custom Wazuh detection rules.
- Alert validation and detection accuracy measurement.
- Automatic case creation from Wazuh alerts to DFIR-IRIS.

1.4 Current Project Status

The project is currently at the **implementation-ready** stage. The lab infrastructure has been designed and prepared, but it has not yet reached the **detection-evaluation** stage. The next phase should focus on generating controlled attack traffic, collecting logs, identifying useful fields, and developing Wazuh rules based on real attack patterns.

Table 1.1: Current implementation status

Task	Status	Notes
Docker lab implementation	Completed	crAPI, Wazuh, and DFIR-IRIS stacks prepared
Network segmentation	Completed	External, DMZ, backend, database, and SOC networks defined
Wazuh infrastructure	Completed	Manager, indexer, and dashboard configured
Wazuh agents	Prepared	Agents mapped to web, backend, and database layers
Pentesting	Pending	To be done in the next phase
Attack-pattern log collection	Pending	Requires controlled attacks first
Custom Wazuh rules	Pending	Requires real logs and field extraction
Detection evaluation	Pending	Requires alerts from custom rules
IRIS integration	Pending	Case creation workflow to be added later

Chapter 2

Implemented Lab Architecture

2.1 Architecture Summary

The implemented lab is designed as a small SOC environment for monitoring an intentionally vulnerable API application. The target application is crAPI, which represents a modern microservice-based API system. The monitoring platform is Wazuh, which provides centralized log collection, rule-based detection, indexing, and dashboard visualization. DFIR-IRIS is included as a separate incident response platform for future case management.

The architecture follows a layered approach. The external user or attacker reaches the lab through a router container. The DMZ layer contains the web gateway and Mail-Hog interface. The backend layer contains crAPI application services. The database layer contains PostgreSQL, MongoDB, and ChromaDB. The SOC monitoring layer contains Wazuh components and Wazuh agents.

2.2 Lab Components

Figure 2.1 presents the current lab components. This diagram represents what has been implemented at the infrastructure level. It does not represent completed detection rules or completed pentest results.

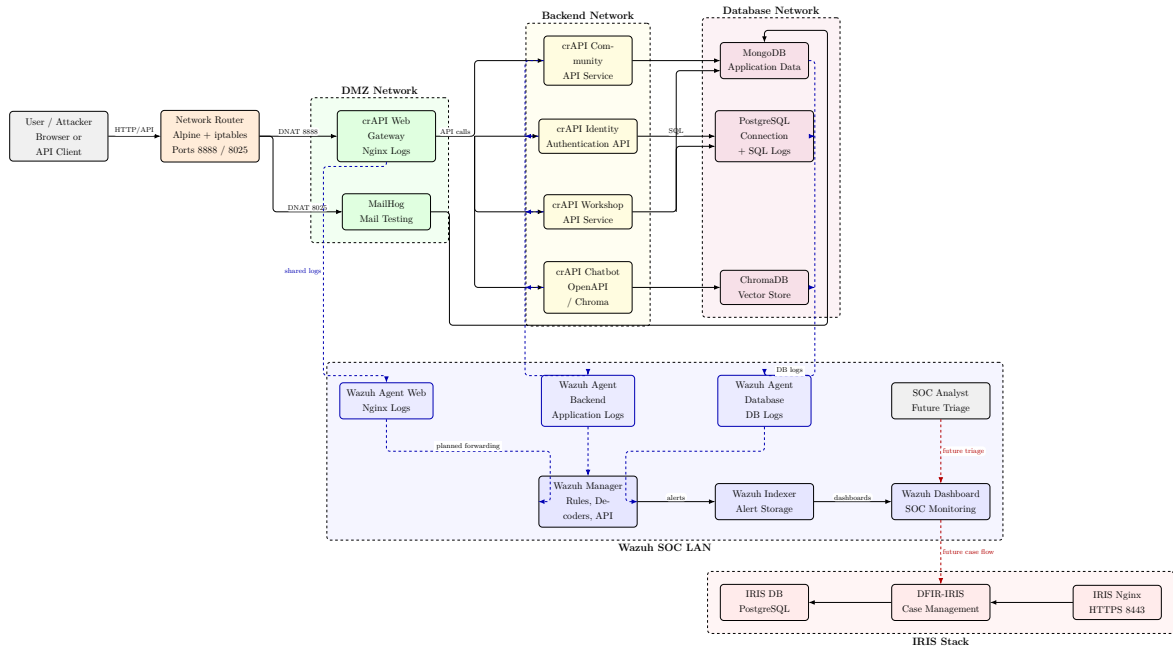


Figure 2.1: Implemented lab components for the Docker-based SOC laboratory.

2.3 Lab Simulation Status

At this stage, the lab simulation workflow is planned but not fully executed. Figure 2.2 shows the intended workflow for the next phase. The current report uses this workflow as a design plan only. Real attack execution, log collection, rule writing, and detection validation will be documented in later reports.

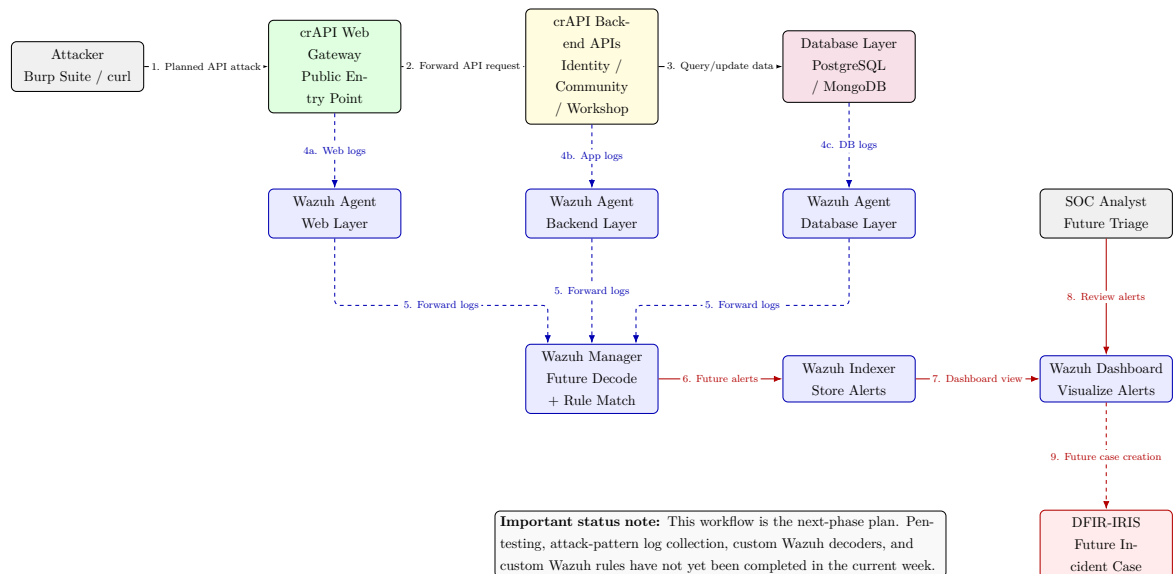


Figure 2.2: Planned lab simulation workflow for the next phase.

Chapter 3

Docker-based Implementation

3.1 crAPI Target Environment

The crAPI environment is implemented as the vulnerable API target. It is organized into several service groups. The web gateway is placed in the DMZ layer. The application services are placed in the backend layer. The data services are placed in the database layer. This structure makes the lab closer to a real enterprise architecture than a flat single-network deployment.

Table 3.1: Implemented crAPI service grouping

Layer	Services	Purpose
External access	network-router	Exposes selected ports and forwards traffic using iptables
DMZ	crAPI web, MailHog	Web entry point and mail testing interface
Backend	identity, community, workshop, chatbot, API gateway	Main API services and business logic
Database	PostgreSQL, MongoDB, ChromaDB	Persistent data, application state, and vector storage
SOC monitoring	Wazuh agents	Collect logs from selected layers

3.2 Network Segmentation

The Docker configuration uses separate networks for different trust zones. The DMZ, backend, and database networks are configured as internal bridge networks. This is

useful for controlling direct access between layers. The Wazuh SOC LAN is attached to monitoring agents so that logs can be forwarded to Wazuh components.

Table 3.2: Implemented Docker network zones

Network	Type	Role
external_net	Bridge	Simulates external access to the lab
dmz_net	Internal bridge	Hosts web-facing services
backend_net	Internal bridge	Hosts application services
database_net	Internal bridge	Hosts database services
wazuh_soc_lan	External network	Connects monitoring agents to the Wazuh stack

3.3 Wazuh SIEM/XDR Stack

The Wazuh stack includes Wazuh Manager, Wazuh Indexer, and Wazuh Dashboard. The manager is responsible for receiving agent data, applying decoders and rules, and generating alerts. The indexer stores security events. The dashboard provides a web interface for monitoring and analysis.

In the current phase, Wazuh infrastructure is prepared. However, custom API-attack rules have not yet been written because real attack-pattern logs have not been collected yet.

Table 3.3: Wazuh components prepared in the current implementation

Component	Image Version	Current Role
Wazuh Manager	4.14.4	Central log analysis and future rule matching
Wazuh Indexer	4.14.4	Alert indexing and storage
Wazuh Dashboard	4.14.4	SOC monitoring interface
Wazuh Agents	4.14.4	Future log collection from web, backend, and database layers

3.4 Wazuh Agent Mapping

Three Wazuh agent containers are planned for different log sources. This design provides better separation between log types and makes future detection engineering easier.

For example, a web attack can be traced from Nginx logs, while backend and database logs can provide additional context.

Table 3.4: Wazuh agent and log source mapping

Agent	Mapped logs	Future detection value
wazuh-agent-web	Nginx logs from crAPI web	API paths, HTTP status codes, request patterns
wazuh-agent-backend	Identity, community, and workshop logs	Authentication events, business logic errors, backend API traces
wazuh-agent-database	PostgreSQL and MongoDB logs	Database connections, SQL statements, database-side anomalies

3.5 DFIR-IRIS Case Management Stack

DFIR-IRIS is included for future incident response workflow. The prepared stack includes IRIS application, Nginx, PostgreSQL database, worker service, and RabbitMQ broker. In the current week, IRIS is treated as an infrastructure component. Automatic integration between Wazuh and IRIS has not yet been completed.

Table 3.5: DFIR-IRIS components prepared for future case management

Component	Role	Current status
IRIS app	Web application and API	Prepared
IRIS Nginx	HTTPS reverse proxy	Prepared on port 8443
IRIS database	PostgreSQL storage	Prepared
RabbitMQ	Celery broker	Prepared
Worker	Background tasks	Prepared
Wazuh-to-IRIS automation	Case creation workflow	Pending

Chapter 4

Current Validation and Limitations

4.1 Infrastructure Validation Performed

The current validation focuses on whether the infrastructure components can be defined and started through Docker Compose. The validation does not include offensive testing or detection accuracy evaluation.

The expected infrastructure validation checklist includes:

- Confirm that required Docker networks exist.
- Confirm that crAPI service containers can start.
- Confirm that Wazuh Manager, Indexer, and Dashboard containers can start.
- Confirm that Wazuh agents can resolve the Wazuh Manager hostname.
- Confirm that shared log volumes are mounted into agent containers.
- Confirm that DFIR-IRIS services can start and expose HTTPS access.

4.2 Known Implementation Issues

Several implementation issues can appear in this stage. One important issue is port binding. For example, binding a service to a specific host IP address such as `10.10.1.1:8888` requires that the host actually owns that IP address. If the IP address is not assigned to a local interface, Docker cannot bind the port and the container start process fails.

Another possible issue is external network dependency. The crAPI Wazuh agent configuration uses `wazuh_soc_lan` as an external network. Therefore, this network must exist before running the compose file. If the Wazuh stack creates a different network name, the crAPI compose file must be updated accordingly.

4.3 Current Limitations

The current report has the following limitations:

- No attack traffic has been generated yet.
- No malicious API requests have been captured yet.
- No baseline log dataset has been created yet.
- No custom Wazuh decoder has been tested yet.
- No Wazuh rule has been written for BOLA or Broken Access Control yet.
- No detection metric such as true positive, false positive, or detection rate can be calculated yet.

4.4 Reason for Deferring Pentest and Rule Writing

Pentesting and rule writing should be done after the infrastructure is stable. If the lab is unstable, collected logs may be incomplete or inconsistent. Detection rules written too early may be based on unreliable log formats. Therefore, the correct workflow is:

1. Stabilize the Docker lab.
2. Confirm that log sources are available.
3. Perform controlled attacks.
4. Collect web, backend, and database logs.
5. Identify useful fields and attack patterns.
6. Write decoders and rules.
7. Validate alerts and reduce false positives.

Chapter 5

SIEM Platform Comparison

5.1 Purpose of the Comparison

This project currently uses Wazuh because it is open-source, practical for local deployment, and suitable for academic SOC lab implementation. However, Microsoft Sentinel and Elastic Stack are also important security monitoring platforms. The comparison below helps justify the current tool choice and shows how the project could be extended in the future.

Table 5.1: Comparison between Wazuh, Microsoft Sentinel, and Elastic Stack

Criteria	Wazuh	Microsoft Sentinel	Elastic Stack
Deployment model	Self-hosted, Docker or VM deployment	Cloud-native SIEM on Microsoft Azure	Self-hosted or cloud-managed Elastic deployment
Cost for student lab	Free and open-source	Requires Azure subscription and log ingestion cost	Free basic stack possible, but advanced security features may require license
Best use case	Endpoint monitoring, log analysis, compliance, lab SOC	Enterprise cloud SOC, Microsoft ecosystem monitoring, cloud-scale analytics	Flexible log search, dashboards, analytics, and custom pipelines

Criteria	Wazuh	Microsoft Sentinel	Sen-	Elastic Stack
Detection logic	XML-based rules, decoders, active response	KQL rules, Microsoft intelligence	analytics playbooks, threat	Query-based detection, Security ingest pipelines
Ease of local lab deployment	High	Low for fully local lab	local	Medium
Integration with crAPI lab	Practical through agents and log files	Possible forwarding logs to Azure	through logs to	Practical through Filebeat/Logstash pipelines
Case management	Basic alerting; external tool needed for full case workflow	Integrated with Microsoft security ecosystem	with security	Requires Elastic Security or external case tools
Reason in this project	Chosen as main SIEM/XDR platform	Compared as enterprise cloud reference	SIEM	Compared as flexible log analytics alternative

5.2 Comparison Analysis

Wazuh is the most suitable choice for the current weekly implementation because it can run locally in Docker and does not require cloud subscription cost. It also supports agent-based monitoring, which fits the segmented Docker architecture. Microsoft Sentinel is powerful for enterprise cloud environments, especially when the organization already uses Microsoft Defender, Azure, and Microsoft 365. However, it is less convenient for a fully local student lab because log ingestion and cloud resources can create cost and configuration complexity.

Elastic Stack is strong for log analytics and visualization. It is flexible and useful for custom dashboards. However, when used alone, it requires additional engineering effort to build security-specific rules, case management, and response workflows. In this project, Elastic Stack remains a useful comparison point, while Wazuh is selected as the primary implementation platform.

Chapter 6

Next Phase Plan

6.1 Planned Work for the Next Week

The next week should focus on moving from infrastructure implementation to controlled testing and log collection. The proposed plan is:

1. Fix any remaining Docker networking and port-binding issues.
2. Confirm Wazuh agent enrollment and connectivity.
3. Confirm that web, backend, and database logs are visible inside the corresponding agent containers.
4. Generate normal user activity in crAPI to build a baseline.
5. Perform simple API reconnaissance and authentication testing.
6. Perform controlled Broken Access Control or BOLA test cases.
7. Collect raw logs from all relevant layers.
8. Identify repeated attack patterns in logs.
9. Draft the first Wazuh decoder and rule set.

6.2 Planned Detection Engineering Workflow

The planned workflow for Wazuh detection engineering is:

1. Collect raw logs from Nginx, backend services, and databases.
2. Separate normal logs and suspicious logs.

3. Identify useful fields such as source IP, user ID, endpoint path, HTTP method, status code, object ID, and authentication token state.
4. Write or adjust Wazuh decoders to normalize the logs.
5. Write rules for suspicious API behavior.
6. Test rules using controlled attack logs.
7. Tune severity levels and reduce false positives.
8. Document alert screenshots and SOC triage steps.

6.3 Expected Future Attack Scenarios

The following scenarios are planned for future reports. They are not part of the current completed work:

- Unauthorized access to another user's object by changing object identifiers.
- Repeated access to sequential object IDs.
- Suspicious authentication failures and token abuse.
- Mass enumeration of API endpoints.
- Abnormal database queries triggered by API misuse.

Chapter 7

Conclusion

This week focused on implementing the SOC laboratory infrastructure rather than performing attack simulation or detection engineering. The main achievement is the preparation of a Docker-based environment combining crAPI as the vulnerable API target, Wazuh as the SIEM/XDR monitoring platform, and DFIR-IRIS as the future case management platform. The architecture is segmented into multiple network zones and includes Wazuh agents for web, backend, and database log collection.

The current report intentionally does not include pentesting results, custom Wazuh rules, attack-pattern logs, or detection metrics because those tasks have not yet been completed. The next phase will focus on stabilizing the infrastructure, confirming log visibility, generating controlled API attacks, collecting attack logs, and writing the first custom Wazuh decoders and rules.

By separating implementation progress from detection results, this report provides a more accurate and honest view of the current project status. It also creates a clear foundation for the next weekly report, which can focus on attack simulation, log analysis, and Wazuh rule development.

Appendix A

Configuration Summary

A.1 Wazuh Manager Configuration Summary

The Wazuh Manager container is configured with the Wazuh 4.14.4 image and exposes ports for agent communication, registration, syslog input, and API access. It also mounts certificates and configuration files required for communication with the Wazuh Indexer.

Listing A.1: Important Wazuh Manager ports

```
1 1514/tcp - Agent event forwarding
2 1515/tcp - Agent registration
3 514/udp - Syslog input
4 55000/tcp - Wazuh API
```

A.2 Current Pending Items

Listing A.2: Pending technical tasks

```
1 # Pending tasks for next phase
2 1. Verify Wazuh agent enrollment
3 2. Confirm log files are collected by each agent
4 3. Generate normal crAPI baseline traffic
5 4. Execute controlled API attack scenarios
6 5. Collect raw attack-pattern logs
7 6. Write Wazuh decoders
8 7. Write Wazuh custom rules
9 8. Validate alerts on Wazuh Dashboard
10 9. Integrate selected alerts with DFIR-IRIS cases
```